

Flink 原理详解 - 第 10 讲

Flink SQL、Table API 与流式关系计算

这一讲解决的核心问题：SQL 原本是为“有限表”设计的。Flink 为什么能在一条永远不会结束的数据流上执行 GROUP BY、JOIN、窗口查询，并持续给出正确结果？

传统数据库：静态表

查询开始时数据基本固定
SELECT city, SUM(amount) ...
查询结束 -> 一个有限结果

Flink：动态表 Dynamic Table

新事件不断到来
表的“当前内容”持续变化
连续查询不断维护最新结果

同一条 SQL 语义：任意时刻看结果，都应等价于对“此刻输入快照”做一次批查询

本讲学习目标

- 理解 Dynamic Table（动态表）和 Continuous Query（连续查询）
- 看懂 Changelog：INSERT / UPDATE_BEFORE / UPDATE_AFTER / DELETE
- 理解流式 GROUP BY 为什么会持续更新结果
- 理解窗口 SQL 为什么能产生“最终窗口结果”
- 知道 Streaming JOIN 为什么容易产生大状态
- 区分 Flink SQL、Table API、DataStream API 各自适用的场景

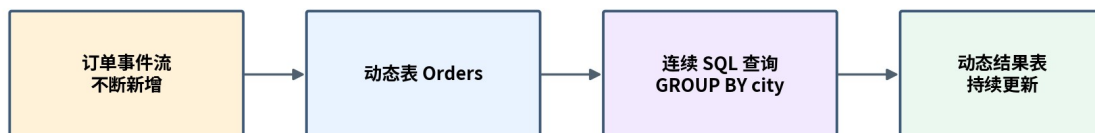
一句话预告：Flink SQL 的关键不是“在流上硬套数据库”，而是把无限事件流解释成一个不断变化的表，再把 SQL 解释成对这个动态表持续维护的查询。

1. SQL 遇到无界流：最大的矛盾是什么？

传统数据库中的表通常可以理解为“此刻已经存在的一组有限记录”。你运行一次 SELECT，数据库读取当前数据，计算完成后返回结果，查询生命周期结束。

但流处理的数据没有“全部到齐”的时刻。例如订单 Kafka Topic 今天不断来数据，明天仍然继续，理论上可以永远运行。于是一个普通问题出现了：如果输入永远不结束，SQL 查询什么时候算完？

关键答案：流式 SQL 通常不是“等待所有数据后再返回一次结果”，而是启动一个 Continuous Query：查询长期运行，每当输入发生变化，就维护结果表的最新状态。



Continuous Query 不“跑完”；每条新数据可能让结果表发生 INSERT / UPDATE / DELETE

流 -> 动态表 -> 连续查询 -> 动态结果表

2. Dynamic Table：动态表

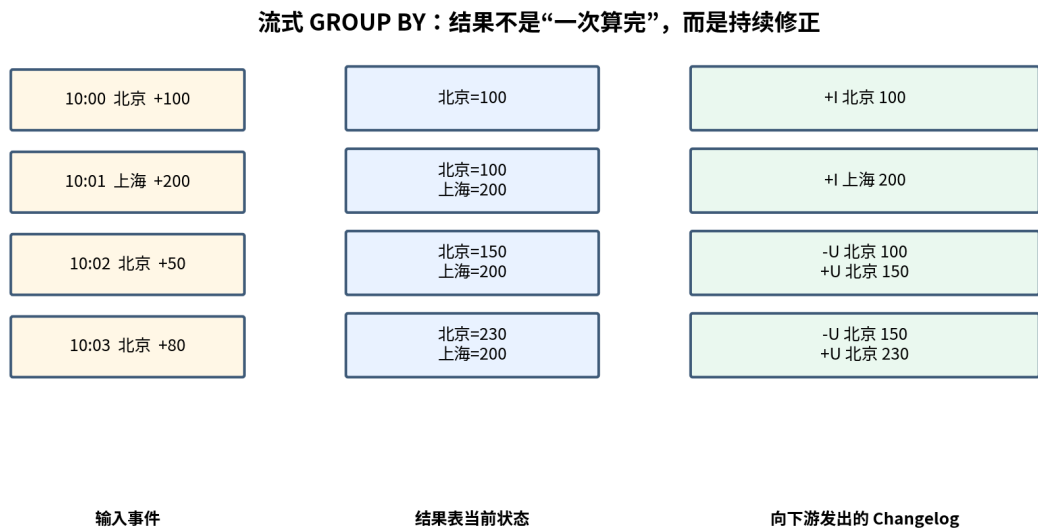
Dynamic Table 是 Flink Table API / SQL 处理流数据的核心逻辑模型。它不是说 Flink 真在内存里维护一张完整数据库表，而是说：从关系语义上，可以把事件流解释成一张“随时间不断变化的表”。

最重要的语义是：在任意时间点 t ，连续查询当前维护出来的结果，应当与“把输入截至 t 的数据做成一个有限快照，再执行同一条批 SQL”在语义上等价。

3. 例子：实时城市销售额为什么会不断改结果？

```
SELECT city, SUM(amount) AS total_amount
FROM Orders
GROUP BY city;
```

假设订单依次到达：北京 100、上海 200、北京 50、北京 80。传统批查询只在所有数据准备好后输出一次；流式查询则需要在数据到来过程中持续维护北京和上海的累计销售额。



这里最容易误解：流式 SQL 的输出不一定是“不断追加新行”。如果以前已经输出过“北京=100”，后面北京又来了 50，那么正确结果必须把北京从 100 改成 150。于是下游需要知道：旧行失效了，新行生效了。

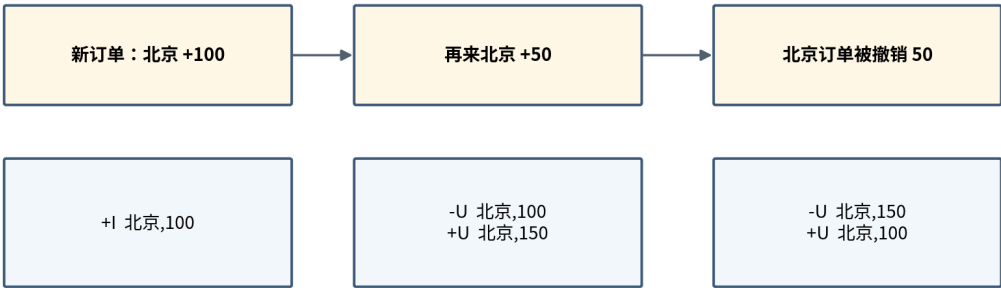
4. Changelog：动态表如何变回“流”？

动态表的变化可以编码成 Changelog Stream（变更日志流）。Flink RowKind 的四种核心变化是：

记号	RowKind	含义
+I	INSERT	插入一条之前不存在的结果行
-U	UPDATE_BEFORE	更新前的旧值，表示这条旧结果需要撤回
+U	UPDATE_AFTER	更新后的新值
-D	DELETE	删除一条结果行

5. Changelog 不只是“内部细节”

例：实时统计每个城市销售额



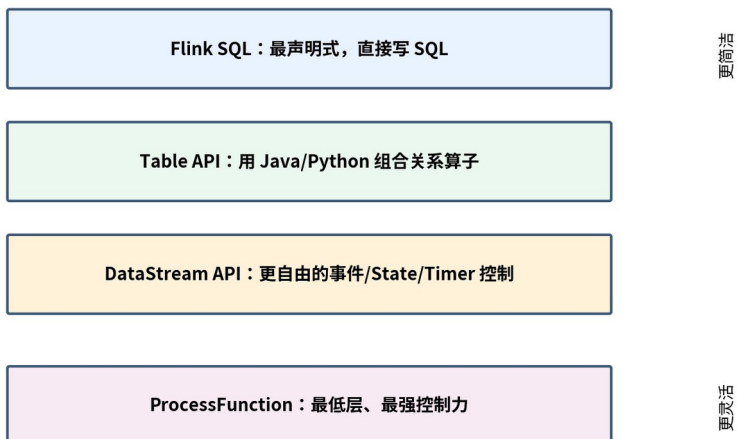
RowKind : +I INSERT, -U UPDATE_BEFORE, +U UPDATE_AFTER, -D DELETE

为什么你必须理解 Changelog? 因为最终结果要写到 Kafka、数据库、文件系统或其他 Sink，而不同 Sink 对“更新”的支持能力不同。

输出形态	直觉	适合的下游
Append-only	只会新增，不修改历史结果	普通 Kafka Topic、日志、文件追加
Upsert	按主键覆盖旧值	Upsert Kafka、带主键数据库表
Retract / Full Changelog	显式撤回旧值再给新值	能理解完整变更语义的下游

例子：实时 GROUP BY 往往不是纯 append-only。你不能简单把“北京=100、北京=150、北京=230”当成三条独立最终记录；下游必须知道后两条是在更新同一个逻辑结果。

6. SQL、Table API、DataStream API 是三套完全不同的引擎吗？



不是。Table API 与 SQL 共享同一套 Table/SQL 执行引擎。它们主要区别在“怎样表达逻辑”：SQL 用 SQL 文本，Table API 用语言内嵌的关系操作。DataStream API 则给你更自由的事件级控制。

7. TableEnvironment：Table/SQL 世界的入口

在 Flink Table API / SQL 中，TableEnvironment 承担了注册表、执行 SQL、管理 Catalog、规划查询等职责。零基础阶段先把它理解成“SQL 世界的控制中心”。

```
from pyflink.table import EnvironmentSettings, TableEnvironment

settings = EnvironmentSettings.in_streaming_mode()
t_env = TableEnvironment.create(settings)
```

表通常不是凭空出现的，而是由 Connector 把外部系统映射成逻辑表。例如 Kafka Topic 可以被声明成一张 Orders 表。

```
CREATE TABLE Orders (
  order_id STRING,
  city STRING,
  amount DECIMAL(10,2),
  event_time TIMESTAMP(3),
  WATERMARK FOR event_time AS event_time - INTERVAL '5' SECOND
) WITH (
  'connector' = 'kafka',
  ...
);
```

别把 CREATE TABLE 理解成 MySQL 建表：在 Flink SQL 中，CREATE TABLE 很多时候是在 Catalog 中声明“这张逻辑表如何连接外部数据源/数据汇”。真正数据仍可能存放在 Kafka、文件、数据库等系统中。

8. Schema 为什么重要？

- 字段名让 SQL 能按列引用数据
- 数据类型决定 SUM、JOIN、比较、时间运算是否合法
- 主键声明会影响 Upsert/Changelog 语义
- Event Time / Watermark 声明决定时间窗口怎样推进

9. 流式 GROUP BY 为什么需要 State ？

还是这条 SQL：

```
SELECT city, SUM(amount) AS total_amount
FROM Orders
GROUP BY city;
```

北京第 100 万条订单到达时，Flink 不可能重新扫描前面 999,999 条北京订单再求和。它必须记住“北京当前累计值”，新数据来时只做增量更新。

底层对应：逻辑上你写的是 GROUP BY；运行时 Flink 会为分组键维护聚合 State。Key 越多、查询运行越久，状态就可能越大。

场景	状态增长特点	风险
按有限城市 GROUP BY	Key 数量较小且稳定	相对容易控制
按 userId GROUP BY	可能数千万/上亿 Key	State 很大
没有窗口、没有过期策略的长期聚合	结果必须持续记住历史	状态可能长期增长

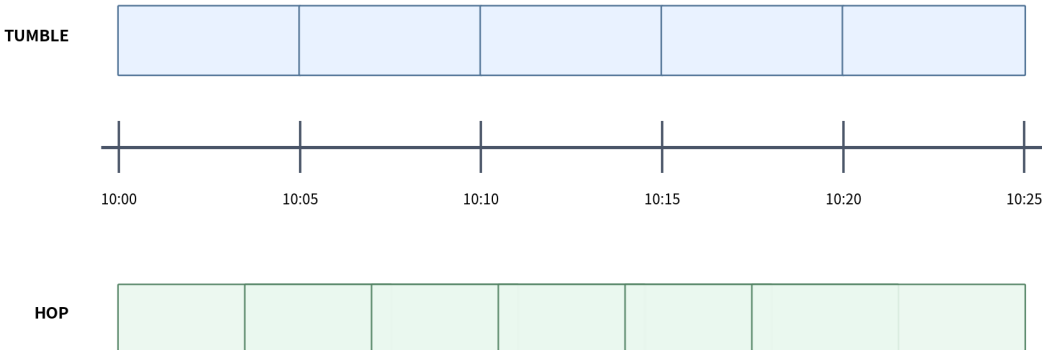
这也解释了一个重要事实：SQL 看起来很短，不代表运行成本很低。声明式 API 把底层复杂度隐藏了，但状态、Shuffle、网络、Checkpoint 仍然真实存在。

10. 为什么窗口 GROUP BY 更容易“收口”？

如果业务问题是“每个城市每 5 分钟销售额”，我们可以用时间窗口把无界问题切成有限片段。窗口结束、Watermark 推进后，旧窗口状态可以被清理。

11. Flink SQL 的 Window TVF

SQL Window TVF 把无界流切成有限“桶”，便于输出最终窗口聚合



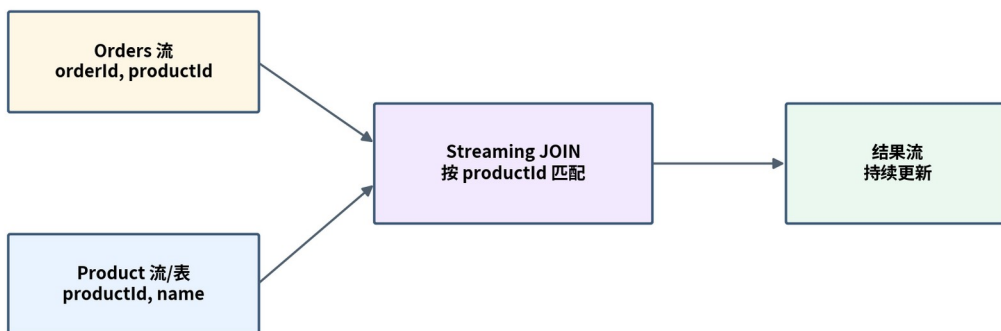
当前 Flink SQL 推荐使用 Windowing Table-Valued Function（Window TVF）来表达时间窗口。常见类型包括 TUMBLE、HOP、CUMULATE，以及流模式下的 SESSION。

```
SELECT
  window_start,
  window_end,
  city,
  SUM(amount) AS total_amount
FROM TABLE(
  TUMBLE(TABLE Orders, DESCRIPTOR(event_time), INTERVAL '5' MINUTES)
)
GROUP BY window_start, window_end, city;
```

这条 SQL 的核心不是语法本身，而是它把无限的 Orders 动态表按照 Event Time 切成一个个有限 5 分钟窗口。每个窗口结束后可以产生一个最终聚合结果，并释放不再需要的窗口状态。

窗口	直觉
TUMBLE	固定大小、不重叠，例如每 5 分钟统计一次
HOP	窗口长度与滑动步长分离，可以重叠
CUMULATE	累计窗口，例如每 5 分钟给出“从本小时开始到现在”的累计值
SESSION	由活动间隔决定边界，适合会话/访问行为

12. Streaming JOIN：为什么比批处理 JOIN 更难？



Regular streaming JOIN 可能需要长期保留两侧历史状态；时间受限的 Join 更容易清理旧状态

批处理 JOIN 的两张表是有限的，读完后就知道全集；流式 JOIN 的两边可能都永远不停。今天来的 Orders 记录，理论上可能要和未来才出现的 Product 变化发生关系。

```

SELECT o.order_id, o.product_id, p.product_name
FROM Orders AS o
JOIN Product AS p
ON o.product_id = p.product_id;
  
```

Regular Streaming Join 的语义很灵活，但代价也明显：为了让未来记录还能和过去记录匹配，系统可能需要长期保留两侧数据的 State。官方文档也特别提示，常规流式 Join 的状态可能无限增长。

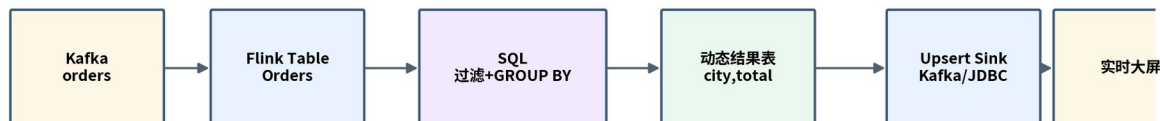
不要机械设置 TTL：TTL 可以限制状态规模，但如果你在业务上仍然需要“很久以前的数据也能与今天的数据匹配”，提前删除状态会直接改变查询结果。TTL 是业务语义和资源消耗之间的取舍，不是免费优化。

13. 时间受限的 Join 为什么更容易控制？

如果业务明确规定“支付事件必须在订单创建后 30 分钟内匹配”，那么系统只需要保留一个有限时间范围的数据，过期后就可以安全清理。因此 Interval Join / Window Join 往往比无限制 Regular Join 更容易控制 State。

14. 一个完整案例：实时城市销售额大屏

实时城市销售额：Flink SQL 不是“隔一会儿查一次”，而是持续维护结果并把变化推给下游



假设 Kafka 中持续写入订单事件，我们要实时维护各城市累计销售额，并把结果写到支持 Upsert 的下游。

```

CREATE TABLE Orders (... ) WITH ('connector'='kafka', ...);

CREATE TABLE CitySales (
  city STRING,
  total_amount DECIMAL(18,2),
  PRIMARY KEY (city) NOT ENFORCED
) WITH ('connector'='upsert-kafka', ...);

INSERT INTO CitySales
SELECT city, SUM(amount)
FROM Orders
WHERE status = 'PAID'
GROUP BY city;
  
```

运行逻辑可以分成 6 步：

- Kafka 新订单变成 Orders 动态表的新增行
- WHERE 过滤无效/非支付订单
- 按 city 做 keyBy/Shuffle（底层概念）
- 为每个 city 维护 SUM 聚合 State
- 结果变化形成 Changelog
- Upsert Sink 按 city 主键覆盖旧结果

为什么需要 PRIMARY KEY：对于 Upsert Sink，下游需要知道“哪一行是同一个逻辑实体”。city 作为主键后，北京从 100 更新为 150 就能覆盖同一个 Key，而不是无限追加多个“北京”记录。

15. Flink SQL / Table API / DataStream：到底怎么选？

问题形态	优先考虑
过滤、投影、GROUP BY、标准 JOIN、窗口聚合	Flink SQL / Table API
需要用程序动态拼装关系查询	Table API
复杂事件级状态机、Timer、任意业务状态	DataStream + ProcessFunction
规则像“3 次失败后 30 秒内异地登录”	CEP 或 DataStream
关系计算为主，但局部需要复杂控制	SQL/Table 与 DataStream 混合

不要把 API 层级理解成“SQL 高级、DataStream 低级，所以 SQL 一定更好”。正确选择标准是：你的问题能否自然表达成关系运算。能表达时，SQL/Table API 更简洁，也能让优化器理解更多语义；不能表达时，再下沉到 DataStream。

16. 常见误区

误区	正确理解
流式 SQL 每隔几秒重新跑一次批 SQL	不是。通常是长期运行的 Continuous Query，增量维护状态和结果
结果只会不断追加	不一定。GROUP BY / JOIN 等会产生更新、撤回、删除
SQL 不需要 State	很多流式聚合和 Join 都依赖大量 State
SQL 把 Shuffle 消灭了	没有。优化器只是选择执行计划，网络重分区仍可能发生
设 TTL 一定安全	TTL 可能改变语义，要先确认业务允许丢弃旧状态
Table API 比 SQL 性能更高	两者共享同一执行引擎，关键在逻辑与优化计划

17. 手工实验：自己模拟一个 Changelog

下面这个 Python 小实验不运行 Flink，但可以帮助你理解“流式 GROUP BY 为什么需要 UPDATE_BEFORE / UPDATE_AFTER”。

```
orders = [
    ('Beijing', 100),
    ('Shanghai', 200),
    ('Beijing', 50),
    ('Beijing', 80),
]

state = {}
for city, amount in orders:
    old = state.get(city)
    new = (old or 0) + amount

    if old is None:
        print('+I', city, new)
    else:
        print('-U', city, old)
        print('+U', city, new)

    state[city] = new
```

你应该观察到：第一次出现北京时输出 INSERT；后面北京金额变化时，输出“撤回旧结果 + 给出新结果”。这就是动态结果表转换成 Changelog Stream 的直觉。

18. 练习题

- 1) 为什么 SELECT * FROM Orders 是 append-only 的概率很高，而 GROUP BY city 往往不是？
- 2) 如果实时统计每个 userId 的累计点击次数，运行一年后 State 可能有什么问题？
- 3) “每 5 分钟每城市销售额”为什么比“从系统启动以来每城市总销售额”更容易清理 State？
- 4) 为什么 Regular Streaming Join 可能需要长期保存两边历史数据？
- 5) 一个 Sink 只支持追加写，但上游 SQL 会产生 UPDATE，你认为会发生什么问题？

参考答案（简版）

- 1) SELECT 通常只是把输入新增行继续向下游传递；GROUP BY 的已有分组结果会被后续数据修改。
- 2) Key 数量可能持续增长，聚合 State 可能越来越大，需要结合窗口、业务生命周期或合理 TTL。
- 3) 窗口有明确时间边界，Watermark 推进后旧窗口可以完成并清理。
- 4) 未来事件可能仍需与过去事件匹配；无限时间范围意味着历史 State 难以自然淘汰。
- 5) Sink 无法表达“旧值失效/新值覆盖”的语义，会得到错误或重复的结果，需要 Upsert/Changelog 能力或改写查询。

19. 本讲统一心智模型

从事件流到 SQL： 事件流先被解释为 Dynamic Table；SQL 作为 Continuous Query 长期运行；查询结果本身仍是 Dynamic Table；结果变化再以 Changelog 的形式流向下游。



把这一讲和前九讲连起来，你会发现 SQL 并没有绕开 Flink 的底层机制：

- GROUP BY 背后仍然需要按 Key 分区和 State
- Window SQL 背后仍然依赖 Event Time、Watermark 与窗口状态
- Streaming JOIN 背后仍然需要双流状态协调
- Changelog 的一致性仍然依赖 Checkpoint 与可靠 Sink
- 性能问题仍然会表现为 Shuffle、State、Backpressure、Checkpoint 等系统问题

20. 当前官方文档核对要点

- 当前 Apache Flink stable 文档版本为 2.3.0。
- Dynamic Table 是 Flink SQL / Table API 流处理的核心逻辑概念；Continuous Query 会不断更新动态结果表。
- RowKind 包含 INSERT、UPDATE_BEFORE、UPDATE_AFTER、DELETE。
- 流式 GROUP BY 可能需要持续增长的 State；官方文档明确提醒长期聚合应关注状态增长。
- Window TVF 当前提供 TUMBLE、HOP、CUMULATE、SESSION 等窗口能力。

下一讲： 第 11 讲建议进入 Flink SQL 的“执行计划与 Connector 实战”：SQL Client / Catalog / Kafka Connector / JDBC / Upsert Kafka / EXPLAIN，以及一条 SQL 从逻辑计划到真实流图如何运行。

官方资料：Apache Flink stable docs - Dynamic Tables, Table API/SQL Overview, Group Aggregation, Windowing TVF, RowKind, Upsert Kafka。